Student application 1

Introduction to AI, ML, and data

Problem statement

An online service receives requests to read or write data. A request is one job sent to the service; latency is the time it takes to finish. The service team wants a warning at arrival if a request will take more than 110 milliseconds (ms). At that moment the finishing time is still unknown. A warning is a prediction, not a measurement of the completed job.

What you are given. You are given code that generates 1,200 independent, fictional past requests. For each request, arrival-time information records the amount of data sent (payload, in MB), the number of jobs already waiting (queue), whether reusable data is in fast storage (cache: 1 for a hit, 0 for a miss), and the operation (endpoint: read or write). The completed request supplies its measured latency and therefore its label: 1 for more than 110 ms, otherwise 0. Some payload measurements are missing. The supplied split reserves 720 rows for learning, 240 for checking choices, and 240 for final assessment.

Concrete example. Illustration, not an extra dataset row: a request arrives with payload 4 MB, queue 3, cache 0, and endpoint read. These four values can inform a warning. If it later finishes in 125 ms, its recorded label is 1; if it finishes in 90 ms, its label is 0. The eventual time cannot be used to predict itself at arrival.

The question to answer. Before building a learned warning model, can we establish trustworthy inputs and measure what a trivial warning rule achieves?

Your assignment

1. Identify the four measurements available at arrival. Exclude the two completed-outcome fields included as traps in the audit table.
2. Prepare numeric training and validation inputs: fill missing values, scale numeric columns, and encode read/write. Learn all preparation settings from training rows only.
3. Fit a baseline that always predicts the most common training label. On validation rows, measure its accuracy and the proportion of genuinely slow requests it catches.

Supplied versus your work. Imports, generated data, fixed row partitions, the preprocessing helper, and check cells are provided. You fill the three marked task cells and write explanations. This application ends with the data audit and baseline; later applications fit richer predictors.

When you are finished. Submit the selected feature names, prepared matrix shapes, validation accuracy and slow-request recall, plus the three written answers. Explain whether the baseline can miss slow requests despite high accuracy. Leave test rows unused; a high score is not required to complete the assignment correctly.

Preparation and notation

Preparation. Allow 60–90 minutes after the lecture. Imports, data, and checks are supplied; no data download or GPU is required. Open the matching notebook:

`01_introduction_and_data_student.ipynb`.

How to work. Run imports and data first. For each task, read its goal, inputs, steps, command explanations, and expected result. Replace each `...` in the starter and remove the intentional `raise NotImplementedError(...)` line. Run the following check. Correct your code if it fails; keep checks, partitions, and seeds unchanged.

Notation. `tr` means training, `va` validation, and `te` test. `X` denotes raw features; `Z` denotes prepared numeric features. A shape `(rows, columns)` describes a matrix; `(rows,)` describes a vector. Class labels are 0 or 1; probabilities range from 0 to 1.

Variable dictionary

Name	Meaning
`X`	All 1,200 requests, with four arrival-time feature columns: payload (MB), queue (count), cache (0/1), endpoint (read/write).
`y`	One label per request: 1 if its eventual latency exceeds 110 ms, otherwise 0.
`train, valid, test`	Arrays of row positions. They contain 720, 240, and 240 positions, respectively.
`Xtr, Xva, Xte`	Raw feature tables for training, validation, and test. The suffixes tr, va, te identify the partition.
`ytr, yva, yte`	Matching target labels, in the same row order as each feature table.
`raw`	A table created before task 1: X plus final_latency and final_status, two deliberately illegal prediction-time features.
`Ztr, Zva`	Numeric feature matrices AFTER preprocessing, for training and validation. Zva is not a label array or a new split. It will have 240 rows and 5 columns.
`prep`	The fitted preprocessing object. It stores training medians, means, standard deviations, and endpoint categories.

Keep the experiment valid

Creating a model sets its options. `fit` learns and stores state; `predict`, `predict_proba`, and `transform` apply that state. Training labels belong in fitting; validation and test labels belong in scoring. For request data, `preprocessing()` creates an unfitted transformer: fitting learns numeric medians for missing values, means and standard deviations for scaling, and endpoint categories for 0/1 indicators. Transformation reuses these choices; an unknown endpoint gets zero in both indicator columns. Record fixed or validation-selected choices before inspecting test. Application 1 uses validation only. Reusing the service test set across these classroom applications is not independent new confirmation.

Task 1. Identify legal features

Goal. Choose features that exist before request execution; do not let the target leak into the inputs.

Inputs. Use the supplied raw table. Its columns are payload, queue, cache, endpoint, final_latency, and final_status.

1. Read each column as an arrival-time measurement or a later outcome. The target depends on final latency, so later outcome columns must be excluded.
2. Create a Python list named legal_features containing exactly the four arrival-time column names as strings.
3. Run the supplied check. Then write one sentence explaining why each excluded column would give the model unavailable information.

Commands and their purpose

`raw.columns`

Returns the table column names; it does not change the table.

`raw[legal_features]`

Selects the columns named in a list, retaining all rows. Square brackets here select columns, not a train/test split.

`set(legal_features)`

Creates an unordered collection used by the supplied check to compare column names. It does not preprocess data.

Starter code

```
# Choose column names, not their observed values.
legal_features = [...]
```

Expected result. legal_features is a list of four strings. Neither final_latency nor final_status is present. No model has been fitted yet.

If you get stuck. If the check fails, inspect spelling and column meaning. Do not remove the check or include a later outcome merely because it improves accuracy.

Task 2. Fit preprocessing

Goal. Create Zva: the validation feature matrix transformed using statistics learned from training only.

Inputs. Use raw, legal_features, and the supplied train/valid row-position arrays. Create Ztr as the corresponding prepared training matrix.

1. Call preprocessing() to create a fresh, unfitted transformer and store it in prep.
2. Select training rows with raw.iloc[train], then legal columns with [legal_features]. Call prep.fit_transform on that table and assign its returned matrix to Ztr.
3. Select validation rows and the same columns. Call prep.transform (not fit_transform) and assign the result to Zva.
4. Run the check and print Ztr.shape and Zva.shape. Explain why four input columns become five prepared columns.

Commands and their purpose

`preprocessing()`

A supplied helper that creates numeric median imputation followed by scaling, plus one-hot encoding of endpoint. Calling it creates an object; it does not yet learn any statistics.

`raw.iloc[train]`

Selects rows by integer position. The train array was provided by the split; do not replace it by all rows.

`prep.fit_transform(table)`

Learns transformation statistics from table, stores them in prep, and returns the transformed numeric matrix.

`prep.transform(table)`

Applies already learned transformations without updating them. This is the correct operation for validation.

`matrix.shape`

Reports (number of rows, number of columns). The validation output should be (240, 5): three numeric columns plus two endpoint indicators.

Starter code

```
prep = preprocessing()
Ztr = prep.fit_transform(...)
Zva = prep.transform(...)
```

Expected result. Ztr has shape (720, 5); Zva has shape (240, 5). Every entry is finite. Both matrices use exactly the same learned transformation.

If you get stuck. Do not overwrite prep by fitting it again on validation. A missing payload is replaced by the training median, not by a validation statistic.

Task 3. Measure the baseline

Goal. Measure how useful always predicting the training majority class is on new validation requests.

Inputs. Use Xtr, ytr for fitting and Xva, yva for validation. This dummy model ignores feature values, so it can take the raw tables directly; it does not need Zva.

1. Create DummyClassifier(strategy="most_frequent") and store it in baseline.
2. Call baseline.fit(Xtr, ytr) to learn which training label is most common.
3. Call baseline.predict(Xva) and store the labels in prediction.
4. Run the supplied accuracy and recall report. Explain in words how a strong accuracy can coexist with catching no violations. Leave the test unopened.

Commands and their purpose

`DummyClassifier(strategy="most_frequent")`

Constructs a baseline that will predict the most frequent training label for every future row.

`baseline.fit(Xtr, ytr)`

Finds the majority label from ytr and stores it in baseline; it returns the fitted object.

`baseline.predict(Xva)`

Returns one class label for every validation row without changing fitted state.

`accuracy_score(yva, prediction)`

Fraction of validation labels predicted correctly.

`recall_score(yva, prediction, zero_division=0)`

Fraction of actual violations caught. The explicit zero convention avoids an undefined numeric result when the relevant denominator is zero.

Starter code

```
baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(..., ...)
prediction = baseline.predict(...)
```

Expected result. prediction is a length-240 array of labels, not probabilities. The report includes validation accuracy and event recall.

If you get stuck. If predict reports that the model is not fitted, run the fit line first. Do not fit using yva to make the baseline look better.

Read the results before drawing conclusions

The supplied assert statements stop execution if a basic requirement is violated; they do not prove the entire reasoning is correct. np.isfinite checks that transformed values are neither missing nor infinite. Accuracy concerns all rows; recall focuses on violations. No test score is needed in this first data-audit application.

The notebook provides the data-generation and split cells before task 1, and any additional task setup immediately before its starter. An `assert` stops execution when a condition is false. A passing check verifies that condition, not every scientific claim. `print` writes values as text; `display` renders tables. Read row and column labels before comparing numbers.

Written interpretation

Answer each question in two to four sentences. Support claims with the displayed values, a calculation, or a concrete counterexample.

1. Explain separately why final_latency and final_status are unavailable at the required prediction time.
2. Describe what happens when endpoint is a category not seen in training. Does successful transformation establish reliable predictions?
3. Propose a split for next month's requests from repeated users. Distinguish temporal transfer from transfer to entirely new users.

Submission checklist

1. Complete all three task cells; leave supplied checks and data unchanged.
2. Restart the kernel and run every cell in order. Save the outputs, including any expected warnings, so another reader can follow the evidence.
3. Include the requested shapes, scores, probabilities, or parameter count and numbered written answers. Label units and distinguish validation from test.
4. State what was fixed in advance and what validation selected. Describe a limitation; do not change your choices after seeing test results.

Criterion	Points	Evidence
Implementation	4	Three tasks completed; checks pass and requested outputs exist.
Experiment	2	Training-only fitting; prescribed split and selection rule.
Interpretation	3	Reasoning tied to results, units or probabilities, and limitations.
Reproducibility	1	Fresh run, saved outputs, declared choices and warnings.

Check your understanding. Identify one failure that a single high accuracy or low loss would miss. Describe an additional check that could reveal it.